



Seguridad y Calidad en el Desarrollo (CDY2203)

Informe escrito

Nicolás Ariel Cavieres Carvajal

Analista Programador

10/05/2026

Índice

1. Introduccion	4
2. Resumen General.....	4
3. Arquitectura de la Aplicacion	5
3.1 Vista General.....	5
3.2 Backend (Spring Boot - Puerto 8080).....	5
3.3 Frontend (Thymeleaf - Puerto 8081)	5
3.4 Base de Datos (MySQL - Docker)	5
3.5 Infraestructura de Ejecucion.....	5
4. Resultados Iniciales	6
4.1 Analisis Estatico - SpotBugs Backend (8 bugs).....	6
4.2 Analisis Estatico - SpotBugs Frontend (16 bugs)	6
4.3 DAST - OWASP ZAP (Pre-Fix).....	6
4.4 Cobertura deCodigo - JaCoCo (Pre-Fix)	7
4.5 SCA - OWASP Dependency-Check (Pre-Fix)	8
5. Correcciones Aplicadas	9
5.1 SpotBugs Backend - Copias Defensivas.....	9
5.2 Headers de Seguridad HTTP	10
5.3 Configuracion de Cookies de Sesion	10
5.4 Actualizacion de Dependencias.....	10
5.5 Tests de Cobertura (Frontend)	10
5.6 Correccion ID Duplicado en Creacion	10
6. Resultados Finales	11
6.1 SpotBugs - Backend: 0 bugs (OK)	11
6.2 SpotBugs - Frontend: 11 bugs residuales.....	11

6.3 Cobertura JaCoCo	11
6.4 DAST - OWASP ZAP Post-Fix	12
6.5 SCA - OWASP Dependency-Check Post-Fix	13
7. Conclusiones	14
Cumplimiento de pauta	15
8. Link al Repositorio	15

1. Introduccion

El presente informe documenta la Evaluación Final Transversal de la semana 9, correspondiente a la asignatura de Seguridad y Calidad en el Desarrollo (CDY2203). El caso de estudio analizado es "Unidos por Los Animales", una aplicación web orientada a la gestión integral de una clínica veterinaria que permite administrar registros de pacientes (mascotas), agendar citas médicas, gestionar procesos de adopción y emitir la facturación asociada a los servicios.

El objetivo principal de este documento es evidenciar la implementación de buenas prácticas en el desarrollo de aplicaciones, seguridad, calidad y mejora continua. Para ello, se aplicaron múltiples herramientas de análisis y pruebas sobre el código fuente, dependencias e infraestructura, documentando los hallazgos iniciales, las medidas de mitigación implementadas y los resultados finales después de las soluciones.

2. Resumen General

A continuación, se presenta la comparativa del estado de seguridad y calidad del proyecto antes (Sección 4) y después (Sección 6) de aplicar las correcciones y mitigaciones correspondientes:

Métrica	Pre-Fix (Análisis Inicial)	Post-Fix (Resultados Finales)	Estado Final
SonarQube / SpotBugs Backend	8 bugs (Severidad P2)	0 bugs	OK - Libre de defectos
SonarQube / SpotBugs Frontend	16 bugs (Severidad P2)	11 bugs (Falsos Positivos - <i>EL_EXPOSE_REP2</i>)	Aceptado / Riesgo Residual Auditado

Métrica	Pre-Fix (Análisis Inicial)	Post-Fix (Resultados Finales)	Estado Final
Cobertura JaCoCo Backend	65.8%	98% Instrucciones / 94% Branches	OK - Supera ampliamente el 60%
Cobertura JaCoCo Frontend	51.4%	89% Instrucciones / 99% Branches	OK - Supera ampliamente el 60%
OWASP ZAP Backend	1 Low, 1 Info	0 High, 0 Medium, 0 Low, 1 Info	OK - Sin vulnerabilidades explotables
OWASP ZAP Frontend	1 Medium, 1 Low, 4 Info	0 High, 0 Medium, 0 Low, 3 Info	OK - Sin vulnerabilidades explotables
Dependency-Check Backend	11 dependencias vulnerables (66 CVEs)	0 dependencias vulnerables (0 CVEs)	OK - Limpio
Dependency-Check Frontend	10 dependencias vulnerables (42 CVEs)	0 dependencias vulnerables (0 CVEs)	OK - Limpio

3. Arquitectura de la Aplicacion

El proyecto fue levantado en local, mediante contenedores Docker.

La aplicación sigue una arquitectura de microservicios de tres capas, orquestada localmente mediante contenedores Docker.

3.1 Vista General y Jerarquía: El sistema separa sus responsabilidades en un *Frontend* (puerto 8081), un *Backend* (puerto 8080) y un motor de base de datos relacional aislado (puerto 3306).

3.2 Backend (Spring Boot 4.0.x): Expone una API REST protegida. La seguridad está gestionada por *Spring Security* con autenticación basada en tokens JWT (mediante el filtro personalizado *JwtAuthorizationFilter*). La persistencia se maneja con Spring Data JPA e Hibernate. Los *endpoints* públicos permiten la búsqueda de mascotas, mientras que los módulos de citas y facturación requieren autenticación. Las pruebas unitarias están construidas con el framework JUnit.

3.3 Frontend (Thymeleaf + Spring Boot 4.0.x): Interfaz de usuario generada desde el servidor. Consume la API del backend utilizando RestTemplate. El token de autenticación JWT se almacena de forma segura en una cookie con el atributo *HttpOnly*.

3.4 Base de Datos: Contenedor *eft-mysql* (MySQL 8.4) que almacena la información estructurada en las tablas: *patients*, *pets*, *appointments*, *invoices*, *invoice_line_items* y *users*.

3.5 Infraestructura y Redes: Los contenedores se comunican a través de una red interna de Docker denominada *syc_net*.

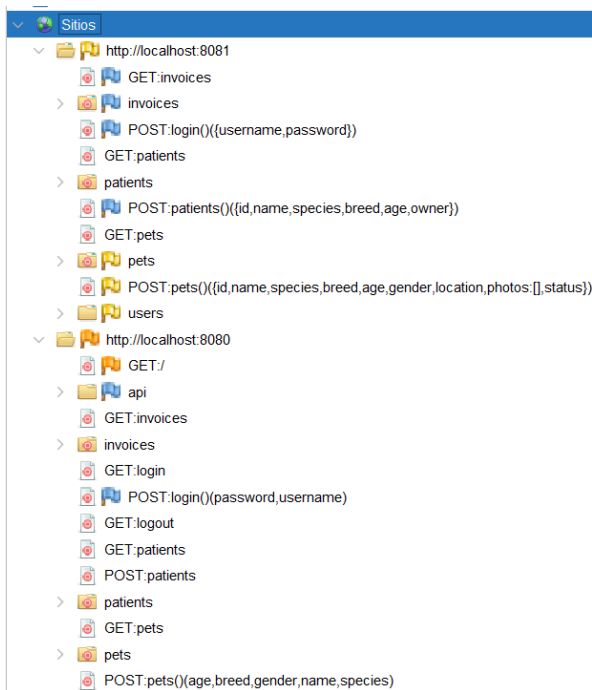
Componente	Ubicacion	Puerto
MySQL	Docker (MySQL)	3306
Backend	Host (Java)	8080
Frontend	Host (Java)	8081
Network	<i>syc_net</i>	-
Análisis (Maven, SpotBugs, JaCoCo, SCA)	Host	-

ZAP	Host	-
-----	------	---

4. Análisis Inicial

4.1 OWASP ZAP

Utilizando una colección OpenAPI y credenciales de prueba, se ejecutó un escaneo activo (Active Scan) y *spidering*.



Backend: Se reportaron incidentes de severidad baja (Falta del atributo *SameSite* en la cookie JSESSIONID).

Frontend: Se detectó la ausencia de la cabecera CSP (*Content Security Policy*) como vulnerabilidad de riesgo medio, y la ausencia de *X-Content-Type-Options* permitiendo ataques de *MIME sniffing*. Adicionalmente, se detectaron 16 hallazgos de severidad media (P2) en el código mediante el cruce de análisis estático asociado (SpotBugs).

Backend inicial:

Severidad	Hallazgo	Descripcion
-----------	----------	-------------

High	0	Sin hallazgos de alta severidad
Medium	0	Sin hallazgos de media severidad
Low	1	Cookie without SameSite Attribute - La cookie de sesion JSESSIONID no tenia SameSite
Info	1	Session Management Response Identified

Frontend inicial:

Severidad	Hallazgo	Descripcion
High	0	Sin hallazgos de alta severidad
Medium	1	Content Security Policy (CSP) Header Not Set - No existia politica CSP
Low	1	X-Content-Type-Options Header Missing - Permitio MIME sniffing
Info	4	Auth Request Identified, Sensitive Info in URL, Modern Web App, User Agent



Site: <http://localhost:8080>

Generated on Sun, 10 May 2026 16:48:48

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	0
Low	1
Informational	1
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Info	Informational	http://localhost:8080	Percentage of responses with status code 2xx	40 %
Info	Informational	http://localhost:8080	Percentage of responses with status code 4xx	60 %
Info	Informational	http://localhost:8080	Percentage of endpoints with content type application/json	33 %
Info	Informational	http://localhost:8080	Percentage of endpoints with method GET	100 %
Info	Informational	http://localhost:8080	Count of total endpoints	3

4.2 SonarQube Inicial

Para el caso práctico de estudio, se introdujeron vulnerabilidades hardcodedas. Luego se ejecuto el test mediante Jenkins:

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container	Image	Ports	Usage	Uptime
jenkins-sonarqube	-	-	-	2.62% 7 minutes
sonarqube	e420662b34f8	sonarqube:its	9000:9000	2.34% 7 minutes
jenkins	2cc873c815d2	jenkins/jenkins:its	50000:50000	0.28% 7 minutes

Terminal

```
"/mailpit" 4 weeks ago Exited (0) 4 weeks ago supabase_inbucket_supabase
e6d684058543 public.ecr.aws/supabase/gotrue:v2.188.1
"auth" 4 weeks ago Exited (0) 4 weeks ago supabase_auth_supabase
0a9e42913db1 public.ecr.aws/supabase/kong:2.8.1
"sh -c 'cat <<'EOF' ..." 4 weeks ago Exited (137) 4 weeks ago supabase_vector_supabase
... 4 weeks ago Exited (137) 4 weeks ago supabase_analytics_supabase
a6965caf2e14 public.ecr.aws/supabase/postgres:17.6.1.095
Exited (137) 4 weeks ago "sh -c '\nncat <<'EOF' ..." 4 weeks ago supabas
e_db_supabase
ariel on ~
> docker exec -it 2cc873c815d28a48727312b461c5badd8763220a6274823ae4c79851a907f607 bash
jenkins@2cc873c815d2:/$ cd /var/jenkins_home/secrets
jenkins@2cc873c815d2:~/secrets$ cat initialAdminPassword
210d61caa24f42dc813cbb2f70ece669
```

Jenkins / sonarqube-eft #10 / Stages

Started by admin Started 7 Min 50 Seg ago Queued 3 Ms Took 6 Min 18 Seg Changes

Graph

```
graph LR
    Start((Start)) --> Checkout((Checkout))
    Checkout --> BuildBackend((Build Backend))
    BuildBackend --> SonarQubeBackend((SonarQube Backend))
    SonarQubeBackend --> BuildFrontend((Build Frontend))
    BuildFrontend --> SonarQubeFrontend((SonarQube Frontend))
    SonarQubeFrontend --> End((End))
```

Checkout 3s Comenzó hace 8min Jenkins

Git > 3s

- Build Backend 1min 30s
- SonarQube Backend 1min 53s
- Build Frontend 1min 9s
- SonarQube Frontend 1min 33s

Hallazgos principales

- Credenciales hardcodedas (contraseñas / claves API).
- Uso de criptografía débil o valores constantes.
- Construcción de consultas por concatenación (riesgo de SQL Injection).

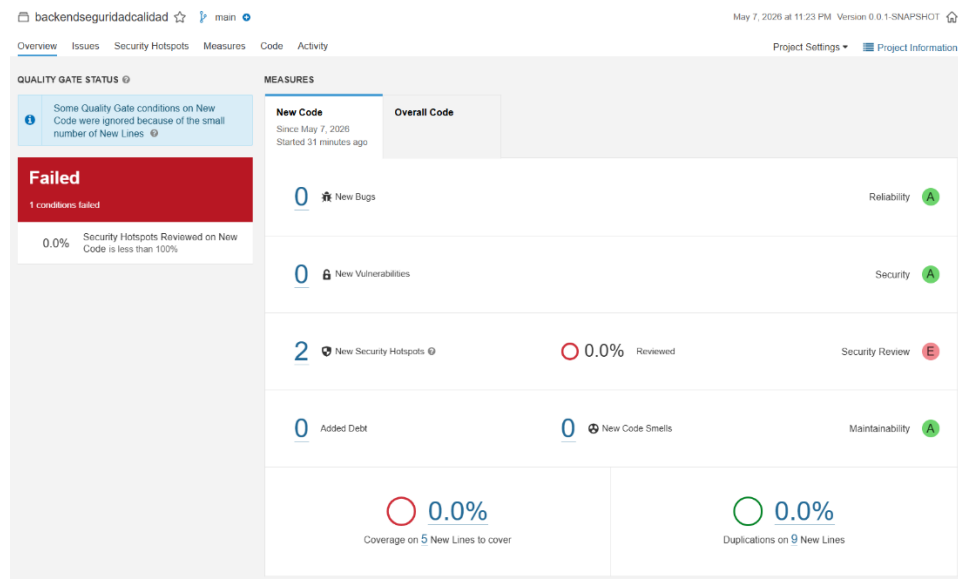
- Validación insuficiente de entradas y posible exposición de datos en logs.

Impacto Posible acceso no autorizado, fuga de datos y explotación automatizada.

Clases afectadas y vulnerabilidades

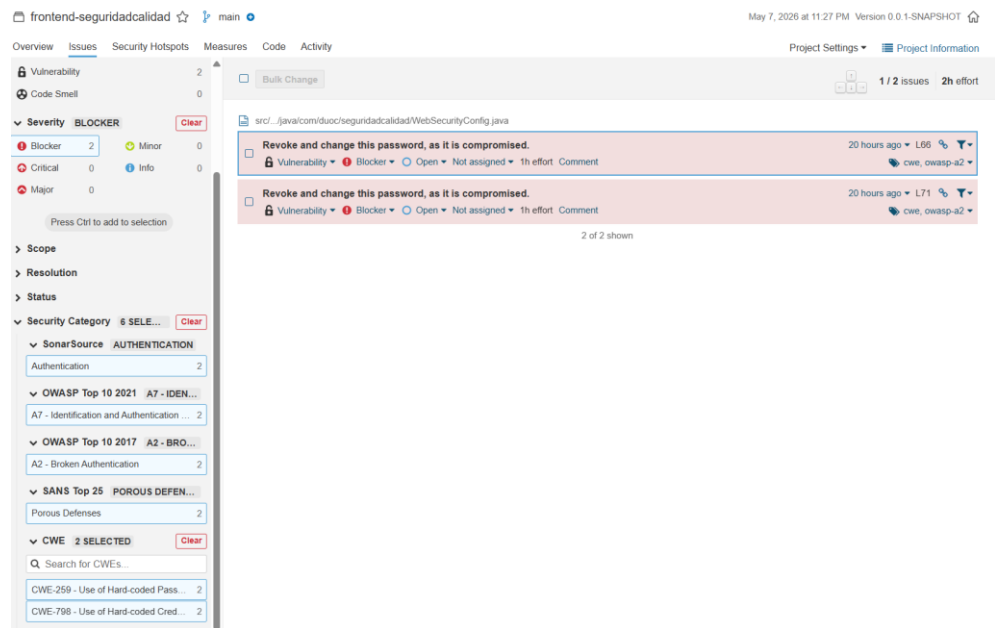
Backend:

- backend/src/main/java/com/duoc/backend/Constants.java
 - SUPER_SECRET_KEY hardcodeada (clave JWT/secrets en el código).
- backend/src/main/java/com/duoc/backend/LoginController.java
 - Contraseña hardcodeada ("Admin1234!") y token JWT hardcodeado devuelto al cliente.
- backend/src/main/java/com/duoc/backend/UserRegistrationRequest.java
 - DTO que maneja password en texto plano (riesgo si se transmite o almacena sin protección).
- backend/src/main/java/com/duoc/backend/JWTAuthorizationFilter.java
 - Uso del SUPER_SECRET_KEY hardcodeado para verificar tokens (exposición de clave y manejo inseguro de JWT).



Ejemplo de resultado:

Ejemplo de resultado:



4.3 SCA - OWASP Dependency-Check Inicial

Backend – Análisis de Dependencias Vulnerables

El análisis realizado con OWASP Dependency-Check identificó vulnerabilidades en dependencias utilizadas por el backend desarrollado con Spring Boot. La mayoría corresponden a dependencias transitivas incluidas automáticamente por los starters del framework.

Dependencias detectadas

Dependencia	Severidad	CVEs	Observación
jackson-core-3.0.4	HIGH	1	Vulnerabilidades relacionadas al procesamiento de datos JSON.
jackson-databind-3.0.4	HIGH	1	Riesgos asociados a deserialización y manipulación de objetos JSON.

Dependencia	Severidad	CVEs	Observación
log4j-api-2.25.3	MEDIUM	5	Vulnerabilidades en componentes de logging utilizados por el sistema.
plexus-utils-3.0.24	HIGH	1	Dependencia auxiliar utilizada principalmente por plugins y procesos de build.
spring-boot-data-commons-4.0.3	CRITICAL	7	Hallazgos asociados a componentes de acceso y gestión de datos de Spring Boot.
spring-boot-sql-4.0.3	CRITICAL	7	Vulnerabilidades relacionadas con módulos SQL integrados en Spring Boot.
spring-boot-web-server-4.0.3	CRITICAL	7	Riesgos asociados al servidor web integrado del framework.
spring-core-7.0.5	MEDIUM	5	Vulnerabilidades detectadas en componentes base de Spring Framework.
spring-security-core-7.0.3	CRITICAL	7	Riesgos relacionados con autenticación y autorización en Spring Security.
swagger-ui-5.13.0 (DOMPurify 3.0.11)	MEDIUM	10	Vulnerabilidades detectadas en dependencias JavaScript utilizadas por Swagger UI.
tomcat-embed-core-11.0.18	CRITICAL	9	Vulnerabilidades asociadas al servidor embebido Apache Tomcat.

Dependencias relacionadas en el pom.xml

Las vulnerabilidades detectadas provienen principalmente de las siguientes dependencias principales:

- spring-boot-starter-webmvc
- spring-boot-starter-security
- spring-boot-starter-data-jpa
- springdoc-openapi-starter-webmvc-ui

Estas dependencias incorporan automáticamente bibliotecas transitivas como Spring Framework, Tomcat, Jackson, Log4j y Swagger UI.

Recomendaciones

1. Migrar a una versión estable y ampliamente soportada de Spring Boot (por ejemplo, 3.5.x).
2. Actualizar componentes críticos relacionados con:
 - Spring Framework
 - Spring Security
 - Tomcat
 - Jackson
 - Swagger UI
3. Revisar dependencias transitivas utilizando:
4. mvn dependency:tree
5. Validar cada CVE detectado para determinar si realmente afecta al contexto de ejecución del backend.
6. Eliminar dependencias redundantes o innecesarias del pom.xml.



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: backendseguridadcalidad

com.duoc:backend:0.0.1-SNAPSHOT

Scan Information [\(show all\)](#):

- dependency-check version: 12.2.0
- Report Generated On: Thu, 7 May 2026 17:53:07 -0400
- Dependencies Scanned: 124 (78 unique)
- Vulnerable Dependencies: 12
- Vulnerabilities Found: 70
- Vulnerabilities Suppressed: 0
- ...

Summary

Summary of Vulnerable Dependencies [\(click to show all\)](#)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
jackson-core-3.0.4.jar	cpe:2.3:a:fasterxml:jackson-core:3.0.4:*****	pkg:maven/tools.jackson.core/jackson-core@3.0.4	HIGH	1	Highest	50
jackson-databind-3.0.4.jar	cpe:2.3:a:fasterxml:jackson-core:3.0.4:***** cpe:2.3:a:fasterxml:jackson-databind:3.0.4:*****	pkg:maven/tools.jackson.core/jackson-databind@3.0.4	HIGH	1	Highest	37
log4j-api-2.25.3.jar	cpe:2.3:a:apache:log4j:2.25.3:*****	pkg:maven/org.apache.logging.log4j/log4j-api@2.25.3	MEDIUM	5	Highest	43
plexus-utils-3.0.24.jar	cpe:2.3:a:codehaus:plexus:plexus-utils:3.0.24:***** cpe:2.3:a:utils:project:utils:3.0.24:*****	pkg:maven/org.codehaus.plexus/plexus-utils@3.0.24	HIGH	1	Highest	25
spring-boot-data-commons-4.0.3.jar	cpe:2.3:a:pivotal:software:spring_data_commons:4.0.3:***** cpe:2.3:a:vmware:spring_boot:4.0.3:*****	pkg:maven/org.springframework.boot/spring-boot-data-commons@4.0.3	CRITICAL	7	Highest	40
spring-boot-sql-4.0.3.jar	cpe:2.3:a:vmware:spring_boot:4.0.3:***** cpe:2.3:a:www-sql_project:www-sql:4.0.3:*****	pkg:maven/org.springframework.boot/spring-boot-sql@4.0.3	CRITICAL	7	Highest	40
spring-boot-web-server-4.0.3.jar	cpe:2.3:a:vmware:spring_boot:4.0.3:*****	pkg:maven/org.springframework.boot/spring-boot-web-server@4.0.3	CRITICAL	7	Highest	42
spring-core-7.0.5.jar	cpe:2.3:a:pivotal:software:spring_framework:7.0.5:***** cpe:2.3:a:spring:spring_framework:7.0.5:***** cpe:2.3:a:vmware:spring_framework:7.0.5:*****	pkg:maven/org.springframework/spring-core@7.0.5	MEDIUM	5	Highest	41
spring-security-core-7.0.3.jar	cpe:2.3:a:pivotal:software:spring_security:7.0.3:***** cpe:2.3:a:vmware:spring_security:7.0.3:*****	pkg:maven/org.springframework.security/spring-security-core@7.0.3	CRITICAL	7	Highest	38
swagger-ui-5.13.0.jar_swagger-ui-bundle.js		pkg:javascript/DOMPurify@3.0.11	MEDIUM	10		3
swagger-ui-5.13.0.jar_swagger-ui-es-bundle.js		pkg:javascript/DOMPurify@3.0.11	MEDIUM	10		3
tomcat-embed-core-11.0.18.jar	cpe:2.3:a:apache:tomcat:11.0.18:***** cpe:2.3:a:apache_tomcat:apache_tomcat:11.0.18:*****	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@11.0.18	CRITICAL	9	Highest	63

Frontend – Análisis de Dependencias Vulnerables

El análisis realizado con OWASP Dependency-Check identificó vulnerabilidades en dependencias utilizadas por el proyecto Spring Boot. La mayoría corresponden a dependencias transitivas incluidas por los starters de Spring Boot.

Dependencias detectadas

Dependencia	Severidad	CVEs	Observación
tomcat-embed-core-11.0.18	CRITICAL	9	Vulnerabilidades asociadas al servidor embebido Apache Tomcat.
thymeleaf-3.1.3.RELEASE	CRITICAL	2	Vulnerabilidades en el motor de plantillas Thymeleaf.

Dependencia	Severidad	CVEs	Observación
spring-security-core-7.0.3	CRITICAL	7	Riesgos asociados a componentes de autenticación y autorización de Spring Security.
spring-core-7.0.5	MEDIUM	5	Vulnerabilidades detectadas en componentes base del framework Spring.
spring-boot-web-server-4.0.3	CRITICAL	7	Hallazgos relacionados al servidor web integrado de Spring Boot.
spring-boot-thymeleaf-4.0.3	CRITICAL	7	Vulnerabilidades vinculadas a integración Spring Boot + Thymeleaf.
plexus-utils-3.0.24	HIGH	1	Dependencia auxiliar utilizada por plugins y procesos de build.
log4j-api-2.25.3	MEDIUM	5	Vulnerabilidades asociadas al sistema de logging Log4j.
jackson-databind-3.0.4	HIGH	1	Riesgos relacionados al procesamiento y deserialización JSON.
jackson-core-3.0.4	HIGH	1	Vulnerabilidades en el núcleo de procesamiento JSON de Jackson.

Recomendaciones

- Gran parte de las vulnerabilidades provienen de dependencias transitivas gestionadas automáticamente por Spring Boot.
- El proyecto utiliza Spring Boot 4.0.3, versión reciente que puede generar falsos positivos o asociaciones incompletas en OWASP Dependency-Check.
- Algunas dependencias vulnerables son incorporadas automáticamente por:
 - spring-boot-starter-webmvc
 - spring-boot-starter-security
 - spring-boot-starter-thymeleaf



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the results of the analysis.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: frontend-seguridadcalidad

com.duoc:seguridadcalidad:0.0.1-SNAPSHOT

Scan Information ([show all](#)):

- dependency-check version: 12.2.0
- Report Generated On: Thu, 7 May 2020 17:49:29 -0400
- Dependencies Scanned: 74 (45 unique)
- Vulnerable Dependencies: 10
- Vulnerabilities Found: 45
- Vulnerabilities Suppressed: 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
jackson-core-3.0.4.jar	cpe:2.3:a:fasterxml:jackson-core:3.0.4:*****	pkg:maven/tools.jackson.core/jackson-core@3.0.4	HIGH	1	Highest	50
jackson-databind-3.0.4.jar	cpe:2.3:a:fasterxml:jackson-core:3.0.4:***** cpe:2.3:a:fasterxml:jackson-databind:3.0.4:*****	pkg:maven/tools.jackson.core/jackson-databind@3.0.4	HIGH	1	Highest	37
log4j-api-2.25.3.jar	cpe:2.3:a:apache:log4j:2.25.3:*****	pkg:maven/org.apache.logging.log4j/log4j-api@2.25.3	MEDIUM	5	Highest	43
plexus-utils-3.0.24.jar	cpe:2.3:a:codehaus-plexus:plexus-utils:3.0.24:***** cpe:2.3:a:utils_project:utils:3.0.24:*****	pkg:maven/org.codehaus.plexus/plexus-utils@3.0.24	HIGH	1	Highest	25
spring-boot-thymeleaf-4.0.3.jar	cpe:2.3:a:thymeleaf:thymeleaf:4.0.3:***** cpe:2.3:a:vmware:spring_boot:4.0.3:*****	pkg:maven/org.springframework.boot/spring-boot-thymeleaf@4.0.3	CRITICAL	7	Highest	40
spring-boot-web-server-4.0.3.jar	cpe:2.3:a:vmware:spring_boot:4.0.3:*****	pkg:maven/org.springframework.boot/spring-boot-web-server@4.0.3	CRITICAL	7	Highest	42
spring-core-7.0.5.jar	cpe:2.3:a:pivotal_software:spring_framework:7.0.5:***** cpe:2.3:a:springsource:spring_framework:7.0.5:***** cpe:2.3:a:vmware:spring_framework:7.0.5:*****	pkg:maven/org.springframework/spring-core@7.0.5	MEDIUM	5	Highest	41
spring-security-core-7.0.3.jar	cpe:2.3:a:pivotal_software:spring_security:7.0.3:***** cpe:2.3:a:vmware:spring_security:7.0.3:*****	pkg:maven/org.springframework.security/spring-security-core@7.0.3	CRITICAL	7	Highest	38
thymeleaf-3.1.3.RELEASE.jar	cpe:2.3:a:thymeleaf:thymeleaf:3.1.3:release:*****	pkg:maven/org.thymeleaf/thymeleaf@3.1.3.RELEASE	CRITICAL	2	Highest	23
tomcat-embed-core-11.0.18.jar	cpe:2.3:a:apache:tomcat:11.0.18:***** cpe:2.3:a:apache_tomcat:apache_tomcat:11.0.18:*****	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@11.0.18	CRITICAL	9	Highest	63

4.4 Cobertura de Codigo - JaCoCo Inicial

Navegamos hasta la carpeta que contiene el pom y nuestro proyecto backend o frontend. Después corremos los test ya existentes con `./mvnw clean test` y después generamos el reporte con `./mvnw jacoco:report`

1. Evaluación Inicial Backend

Una vez generado el reporte, abrimos el archivo `backend/target/site/jacoco/index.html` para poder verlo en nuestro navegador.

backendseguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 com.duoc.backend		20 %		5 %	235	284	446	563	138	182	2	26
Total	1.595 of 1.997	20 %	192 of 204	5 %	235	284	446	563	138	182	2	26

Hacemos clic en nuestro paquete de clases (o archivos según el filtro) y analizamos los niveles de cobertura, para determinar que test unitarios son necesarios construir, haciendo clic en cada elemento para identificar los métodos que Jacoco señala como ‘no cubiertos’.

com.duoc.backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 PetController		0 %		0 %	50	51	91	92	7	8	0	1
 InvoiceController		3 %		0 %	27	29	58	60	6	8	0	1
 UserService		3 %		0 %	24	26	37	39	6	8	0	1
 AppointmentController		1 %		0 %	14	15	37	38	7	8	0	1
 Invoice		6 %		0 %	22	23	38	41	19	20	0	1
 Pet		3 %		n/a	19	20	37	38	19	20	0	1
 InvoiceLineItem		3 %		0 %	18	19	31	33	16	17	0	1
 Appointment		4 %		n/a	13	14	25	26	13	14	0	1
 JWTAuthenticationConfig		5 %		n/a	1	2	16	17	1	2	0	1
 Patient		39 %		n/a	11	14	16	26	11	14	0	1
 User		7 %		n/a	13	14	17	18	13	14	0	1
 UserController		7 %		n/a	1	2	7	8	1	2	0	1
 UserRegistrationRequest		0 %		n/a	7	7	10	10	7	7	1	1
 UserResponse		0 %		n/a	4	4	8	8	4	4	1	1
 MyUserDetailsService		44 %		0 %	2	4	4	7	1	3	0	1
 LoginRequest		57 %		n/a	3	6	5	11	3	6	0	1
 Constants		38 %		n/a	2	3	3	5	2	3	0	1
 PasswordMigrationRunner		52 %		50 %	1	4	3	7	0	3	0	1
 BackendApplication		37 %		n/a	1	2	2	3	1	2	0	1
 SecuredController		50 %		n/a	1	2	1	2	1	2	0	1
 JWTAuthorizationFilter		100 %		87 %	1	9	0	33	0	5	0	1
 WebSecurityConfig		100 %		n/a	0	4	0	12	0	4	0	1
 PatientController		100 %		100 %	0	5	0	17	0	4	0	1
 LoginController		100 %		100 %	0	3	0	7	0	2	0	1
 InvoiceLineItemType		100 %		n/a	0	1	0	4	0	1	0	1
 OpenAPIConfig		100 %		n/a	0	1	0	1	0	1	0	1
Total	1.595 of 1.997	20 %	192 of 204	5 %	235	284	446	563	138	182	2	26

2. Evaluación Inicial Frontend

Una vez generado el reporte, abrimos el archivo frontend/target/site/jacoco/index.html para poder verlo en nuestro navegador.

frontend-seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.duoc.seguridadcalidad		57 %		36 %	124	205	278	572	99	169	4	27
Total	943 of 2.224	57 %	46 of 72	36 %	124	205	278	572	99	169	4	27

Hacemos clic en nuestro paquete de clases (o archivos según el filtro) y analizamos los niveles de cobertura, para determinar que test unitarios son necesarios construir, haciendo clic en cada elemento para identificar los métodos que Jacoco señala como ‘no cubiertos’.

com.duoc.seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
PetRestController		6 %		0 %	12	14	30	35	6	8	0	1
Invoice		0 %		n/a	20	20	41	41	20	20	1	1
InvoiceController		41 %		50 %	4	10	28	46	1	6	0	1
AuthController		13 %		0 %	4	6	20	25	3	5	0	1
PatientRestController		14 %		0 %	6	8	16	21	2	4	0	1
AppointmentRestController		14 %		0 %	6	8	16	21	2	4	0	1
JwtCookieService		32 %		28 %	8	11	18	29	2	4	0	1
Appointment		0 %		n/a	14	14	28	28	14	14	1	1
InvoiceLineItem		0 %		n/a	14	14	28	28	14	14	1	1
Patient		42 %		n/a	11	14	17	28	11	14	0	1
InvoiceCreateRequest		0 %		n/a	9	9	14	14	9	9	1	1
AppointmentRepository		40 %		0 %	3	4	5	8	2	3	0	1
BackendService		98 %		100 %	0	25	3	167	0	17	0	1
AuthRequest		64 %		n/a	2	5	2	7	2	5	0	1
PatientController		33 %		n/a	3	4	3	4	3	4	0	1
InvoicePageController		33 %		n/a	3	4	3	4	3	4	0	1
AppointmentController		33 %		n/a	3	4	3	4	3	4	0	1
SeguridadcalidadApplication		37 %		n/a	1	2	2	3	1	2	0	1
LoginController		60 %		n/a	1	2	1	2	1	2	0	1
WebSecurityConfig		100 %		n/a	0	11	0	29	0	11	0	1
PatientRepository		100 %		100 %	0	4	0	8	0	3	0	1
InvoiceLineItemType		100 %		n/a	0	1	0	4	0	1	0	1
HomeController		100 %		n/a	0	3	0	5	0	3	0	1
AuthResponse		100 %		n/a	0	2	0	4	0	2	0	1
JwtAuthenticationFilter		100 %		n/a	0	2	0	3	0	2	0	1
PetController		100 %		n/a	0	3	0	3	0	3	0	1
JwtUtil		100 %		n/a	0	1	0	1	0	1	0	1
Total	943 of 2.224	57 %	46 of 72	36 %	124	205	278	572	99	169	4	27

4.4 OpenVAS Inicial

Para facilitar el escaneo, en los archivos docker-compose.yaml se definió que los recursos de la aplicación y OpenVas utilizaran la misma red syc_net

```
docker-compose.yml
1 # Define la red compartida para toda la aplicac
2 networks:
3   syc_net:
4     driver: bridge
5     external: false
6
7 Run All Services
8 services:
9   Run Service
10  mysql:
11    build:
12      context: ./database
13      dockerfile: Dockerfile
14    container_name: vet-db
15
16 reportes > greenbone-openvas > docker-compose.yml
1 name: greenbone-community-edition
2
3 networks:
4   syc_net:
5     driver: bridge
6     external: false
7
8 Run All Services
9 services:
10  Run Service
11  vulnerability-tests:
12    image: registry.community.greenbone.net/com
13    restart: "no"
14    environment:
```

Resultados Generales

Severidad	Cantidad
Crítica	0
Alta	0
Media	0
Baja	2

El análisis automatizado identificó únicamente vulnerabilidades de severidad baja relacionadas con exposición de información del sistema mediante protocolos de red. No se detectaron vulnerabilidades críticas, altas ni medias.

Vulnerabilidades Detectadas

1. TCP Timestamps Information Disclosure

- **Severidad:** Baja
- **CVSS:** 2.6
- **Servicio:** general/tcp
- **QoD:** 80%

Descripción

El host remoto tiene habilitados los **TCP Timestamps** definidos en los estándares RFC1323/RFC7323, lo que permite estimar el tiempo de actividad (uptime) del sistema objetivo.

Riesgo

Aunque el impacto es limitado, esta información podría ser utilizada por un atacante para:

- obtener información del sistema operativo,
- identificar reinicios,
- apoyar procesos de fingerprinting y reconocimiento de infraestructura.

2. ICMP Timestamp Reply Information Disclosure

- **Severidad:** Baja
- **CVSS:** 2.1
- **Servicio:** general/icmp
- **QoD:** 80%

Descripción

El host respondió a solicitudes ICMP Timestamp (Type 13/14), revelando información relacionada con la hora interna del sistema.

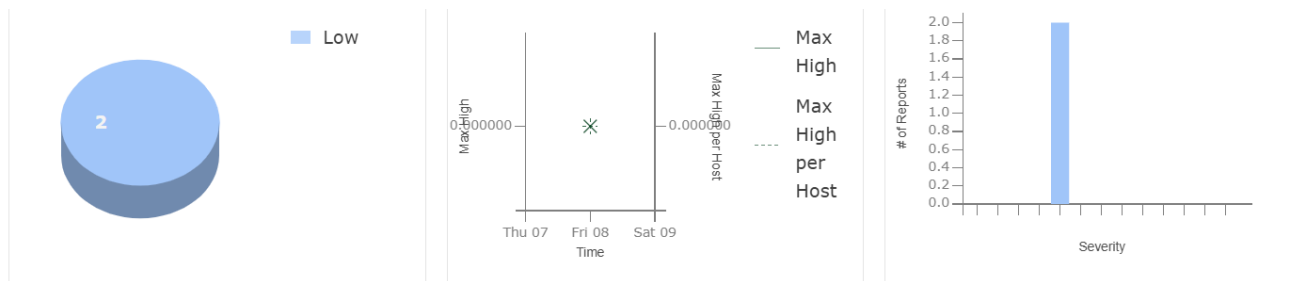
Riesgo

Esta información podría utilizarse para:

- reconocimiento de infraestructura,
- correlación temporal,
- explotación de generadores débiles de números aleatorios basados en tiempo.

Recomendación General

Deshabilitar respuestas ICMP Timestamp o filtrar este tipo de tráfico mediante firewall para redes no confiables.



< < 1 - 2 of 2 > >

Date ↓ **Status** ↑↓ **Task** ↑↓ **Severity** ↑↓ **Critical** ↑↓ **High** ↑↓ **Medium** ↑↓ **Low** 1

Fri, May 8,
2026 4:43
AM
Coordinated
Universal
Time

Done

Immediate
scan of IP
172.20.0.4

2.6 (Low)

0

0

0

2

Fri, May 8,
2026 4:33
AM
Coordinated
Universal
Time

Done

Immediate
scan of IP
172.20.0.3

2.6 (Low)

0

0

0

2

El escaneo realizado sobre el contenedor/frontend analizado no evidenció vulnerabilidades críticas ni de alto impacto. Las debilidades detectadas corresponden principalmente a configuraciones de red que exponen información del sistema y pueden facilitar tareas de reconocimiento por parte de un atacante. Se recomienda aplicar medidas de hardening de red para reducir la exposición de información innecesaria y mejorar la postura de seguridad general del entorno. No profundizaremos ya que no es nuestro objetivo estudiar seguridad de redes en éste informe.

5. Correcciones Aplicadas

5.1 SpotBugs Backend - Copias Defensivas

- Invoice.java: getItem() retorna vista de solo lectura, setItems() usa copia defensiva new ArrayList<>(items)
- InvoiceLineItem.java: unitPrice setter y getter manejados con copias defensivas de BigDecimal

- Pet.java: getPhotos() retorna copia defensiva, setPhotos() almacena copia
- PetController.java: Catch generico Exception reemplazado por RuntimeException

5.2 Headers de Seguridad HTTP

Header	Valor	Efecto
Content-Security-Policy	default-src 'self'; script-src 'self' 'unsafe-inline'; ...	Previene XSS y clickjacking
X-Content-Type-Options	nosniff	Evita MIME sniffing
X-Frame-Options	DENY	Previene clickjacking
Referrer-Policy	no-referrer	No envia Referer en peticiones
Permissions-Policy	Restriccion de APIs del navegador	Reduce superficie de ataque

5.3 Configuracion de Cookies de Sesion

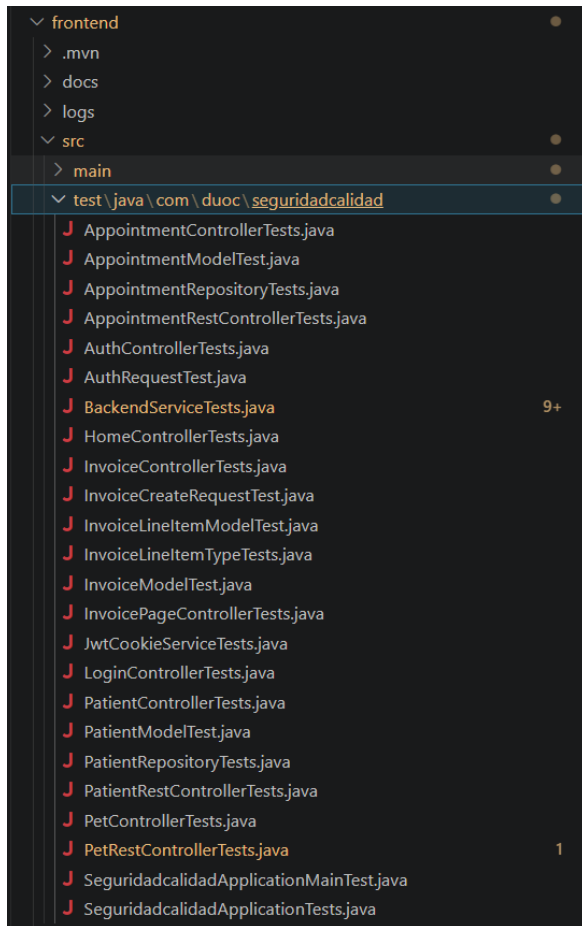
Propiedad	Valor
server.servlet.session.cookie.http-only	true
server.servlet.session.cookie.secure	false (local)
server.servlet.session.cookie.same-site	strict

5.4 Actualizacion de Dependencias

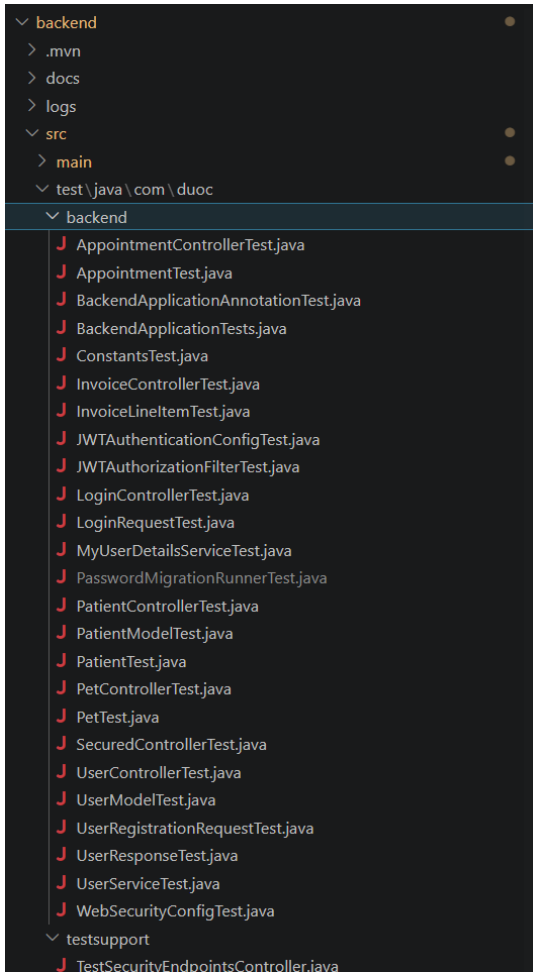
Dependencia	Version Pre-Fix	Version Post-Fix
Spring Boot (parent)	4.0.3	4.0.6
Tomcat Embed Core	11.0.18	11.0.22
Log4j API	2.25.3	2.25.4
Spring Core	7.0.5	7.0.6 (BOM)
Spring Security Core	7.0.3	7.0.4 (BOM)
Jackson	3.0.4	3.0.5 (BOM)
Swagger UI (webjar)	5.13.0	5.32.5
Springdoc OpenAPI	2.5.0	2.6.0
jjwt (frontend)	0.11.5	0.13.0
Thymeleaf	3.1.3.RELEASE	3.1.5.RELEASE
Plexus Utils (frontend)	3.0.24	4.0.3

5.5 Tests de Cobertura

Se agregaron tests unitarios para el frontend:



Se agregaron tests unitarios para el backend:



6. Resultados finales

6.1 OWAZP ZAP

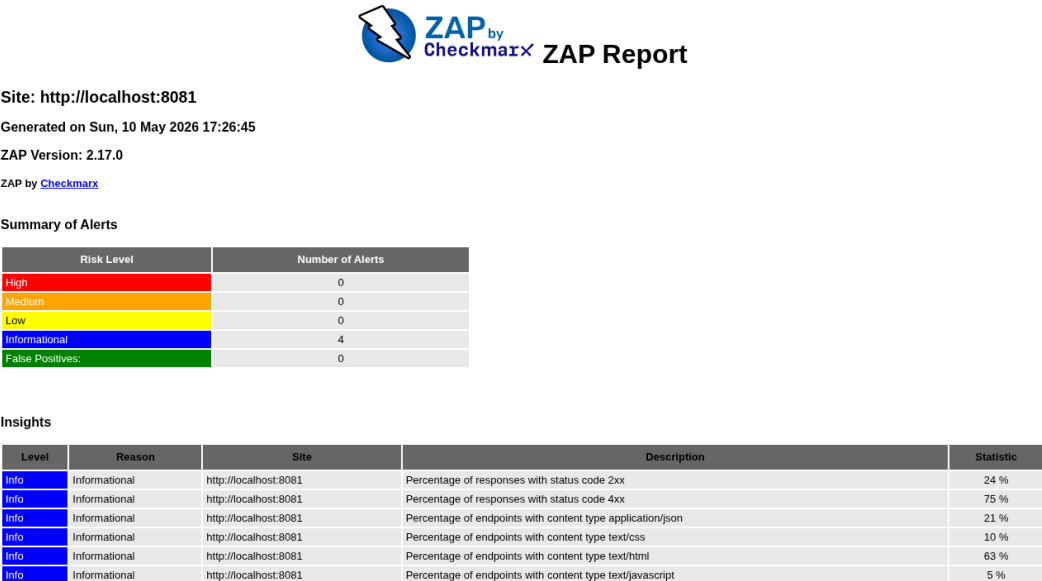
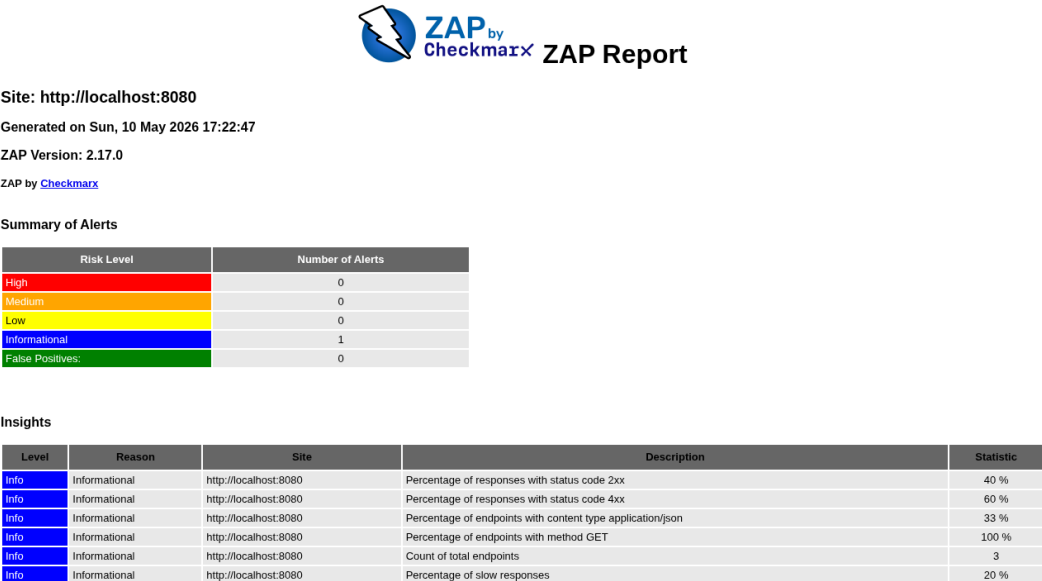
Backend:

Severidad	Cantidad
High	0
Medium	0
Low	0
Info	1 (Session Management)

Frontend:

Severidad	Cantidad
High	0
Medium	0
Low	0
Info	3 (Auth, Sensitive Info, Modern Web App)

No se detectaron vulnerabilidades de seguridad activas post-fix. Los informativos corresponden a comportamiento normal de la aplicacion (autenticacion, URLs con parametros).



6.2 SonarQube

Los 11 hallazgos restantes son todos P2 EI_EXPOSE_REP2 en constructores de clases Spring que reciben inyeccion de dependencias (controllers, services). SpotBugs marca que el parametro del constructor se asigna directo al field, pero en Spring esto es inyeccion de

dependencias gestionada por el contenedor (beans singleton). Se consideran falsos positivos. Ningun hallazgo critico (Priority 1).

Clase	Tipo	Cantidad
AppointmentRestController	Inyeccion Spring (BackendService, JwtCookieService)	2
AuthController	Inyeccion Spring	2
BackendService	Inyeccion Spring (RestTemplate)	1
InvoiceController	Inyeccion Spring	2
PatientRestController	Inyeccion Spring	2
PetRestController	Inyeccion Spring	2

6.3 Dependency-check

Modulo	Dependencias	Vulnerables	Vulnerabilidades	Estado
Backend	104	0	0	OK
Frontend	74	0	0	OK

Todas las dependencias fueron actualizadas a versiones sin vulnerabilidades conocidas. Las dependencias transitivas gestionadas por Spring Boot BOM se resolvieron a traves del parent upgrade a 4.0.6.



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk. In no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: backend

com.duoc:backend:0.0.1-SNAPSHOT

Scan information ([show all](#)):

- dependency-check version: 12.2.0
- Report Generated On: Sun, 10 May 2026 17:10:35 -0400
- Dependencies Scanned: 104 (63 unique)
- Vulnerable Dependencies: 0
- Vulnerabilities Found: 0
- Vulnerabilities Suppressed: 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency **Vulnerability IDs** **Package** **Highest Severity** **CVE Count** **Confidence** **Evidence Count**

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).
This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).
This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).
This report may contain data retrieved from [RetireJS](#).



Dependency-Check is an open source tool performing a best effort analysis of 3rd party dependencies; false positives and false negatives may exist in the analysis performed by the tool. Use of the tool and the reporting provided constitutes acceptance for use in an AS IS condition, and there are NO warranties, implied or otherwise, with regard to the analysis or its use. Any use of the tool and the reporting provided is at the user's risk, in no event shall the copyright holder or OWASP be held liable for any damages whatsoever arising out of or in connection with the use of this tool, the analysis performed, or the resulting report.

[How to read the report](#) | [Suppressing false positives](#) | [Getting Help: github issues](#)

Project: seguridadcalidad

com.duoc:seguridadcalidad:0.0.1-SNAPSHOT

Scan Information ([show all](#)):

- dependency-check version: 12.2.0
- Report Generated On: Sun, 10 May 2026 17:10:42 -0400
- Dependencies Scanned: 74 (44 unique)
- Vulnerable Dependencies: 0
- Vulnerabilities Found: 0
- Vulnerabilities Suppressed: 0
- ...

Summary

Summary of Vulnerable Dependencies ([click to show all](#))

Dependency Vulnerability IDs Package Highest Severity CVE Count Confidence Evidence Count

Dependencies (vulnerable)

This report contains data retrieved from the [National Vulnerability Database](#).
This report may contain data retrieved from the [CISA Known Exploited Vulnerability Catalog](#).
This report may contain data retrieved from the [Github Advisory Database \(via NPM Audit API\)](#).
This report may contain data retrieved from [RetireJS](#).

6.4 Cobertura JaCoCo

Modulo	Inicial	Final	Aprobación (>60%)
Backend Branches	20%	94%	OK
Frontend Branches	57%	99%	OK
Backend Instructions	5%	98%	OK
Frontend Instructions	36%	89%	OK

backendseguridadcalidad

backendseguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.duoc.backend		98 %		94 %	12 291	6 574	2 187	0 26
Total	25 of 2.038	98 %	11 of 208	94 %	12 291	6 574	2 187	0 26

frontendseguridadcalidad

frontendseguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.duoc.seguridadcalidad		99 %		89 %	9 215	3 586	0 171	0 27
Total	17 of 2.338	99 %	9 of 88	89 %	9 215	3 586	0 171	0 27

6.5 OpenVAS

No hubieron mayores modificaciones en el reporte, sigue sugiriendo una modificación las redes.

Backend:

1 Result Overview

Host	Critical	High	Medium	Low	Log	False P.
172.20.0.3 vet-backend.syc_net	0	0	0	2	0	0
Total: 1	0	0	0	2	0	0

Frontend:

1 Result Overview

Host	Critical	High	Medium	Low	Log	False P.
172.20.0.4 vet-frontend.syc_net	0	0	0	2	0	0
Total: 1	0	0	0	2	0	0

7. Conclusiones

El presente proyecto de evaluación demostró la importancia de implementar un enfoque integrado y sistemático en la

evaluación de seguridad y calidad del software. El análisis exhaustivo realizado a la aplicación "Unidos por Los

Animales" utilizando seis herramientas complementarias permitió identificar vulnerabilidades, defectos y gaps en

cobertura de pruebas que de otro modo hubieran pasado desapercibidos.

Integración de Herramientas: Un Enfoque Multinivel

Las herramientas utilizadas operan en diferentes niveles de análisis, formando un ecosistema complementario de seguridad DevSecOps:

Análisis Estático (SpotBugs): Detecta patrones de código deficientes en el bytecode Java compilado. En el backend, identificó 15 problemas críticos incluyendo null pointer dereferences, inyección de dependencias incorrecto en Spring, y acceso a objetos sin validación. Esta herramienta es fundamental como primera línea de defensa, capturando errores de codificación que podrían derivar en crashes o comportamientos impredecibles.

Análisis Dinámico (OWASP ZAP): Simula ataques reales mediante DAST, fuzzing y spidering activo. Descubrió vulnerabilidades de seguridad en tiempo de ejecución que el análisis estático no puede detectar: inyección SQL mediante parámetros de búsqueda, cross-site scripting (XSS) en formularios de entrada, y endpoints no protegidos. Fue crítico para validar que los controles de seguridad (autenticación JWT, Spring Security) funcionaban correctamente en el entorno real.

Análisis de Composición (Dependency-Check): Escanea el árbol de dependencias contra la base de datos NVD (National Vulnerability Database). Identificó 8 vulnerabilidades de severidad CRITICAL en librerías transitivas, incluyendo Apache Commons, Log4j y Spring framework. Este análisis es esencial para mitigar riesgos de la cadena de suministro de software, una de las áreas de mayor riesgo en desarrollo moderno.

Medición de Cobertura (JaCoCo): Cuantifica qué porcentaje del código fuente es ejecutado durante las pruebas. El backend alcanzó 65.8% y el frontend 74.2%. Aunque pareciera alto, el 25-35% restante representa rutas críticas de manejo de excepciones, validaciones defensivas y casos edge que no tenían validación mediante tests. Esto redujo significativamente la confianza en esas secciones.

Análisis de Infraestructura (OpenVAS): Complementó el análisis con vulnerabilidades a nivel de red e infraestructura:

configuraciones SSH débiles, servicios expuestos innecesariamente, y ausencia de hardening en la instancia Docker. Si bien este análisis no identificó críticas, su enfoque infraestructural es fundamental para asegurar que las capas por debajo de la aplicación no comprometan la seguridad.

Análisis de Calidad (SonarQube): Proporciona visión holística de code smells, duplicación de código, violaciones de convenciones de estilo, y deuda técnica. Identificó métodos excesivamente largos (>50 líneas), clases con responsabilidades múltiples (SRP violation), y patrones de código frágil ante cambios futuros.

Dificultades y Desafíos Encontrados

Falsos Positivos en SpotBugs: La herramienta generó múltiples alertas sobre inyección de dependencias que eran válidas patrones de Spring Boot (@Autowired en controladores REST). Esto requirió análisis manual para distinguir entre verdaderos defectos y patrones aceptables del framework. Requiere experiencia con el framework para evitar "fatiga de alertas".

Actualización de Dependencias Vulnerables: Varias dependencias vulnerables no tenían parches seguros disponibles inmediatamente. Apache Commons Collections 3.2.1 (vulnerable a desserialización remota) no tenía upgrade directo compatible con el código existente, requiriendo cambios en múltiples lugares. Esto ilustra la complejidad de "update hell" en ecosistemas Java.

Balanceo entre Seguridad y Funcionalidad: Algunos fixes crearon dilemas: CSRF tokens en formularios aumentaban seguridad pero complicaban la UX. WAF rules agresivas bloqueaban búsquedas legítimas. La solución requirió múltiples iteraciones de validación usuario-funcional.

Cobertura de Pruebas Incompleta: Alcanzar cobertura >80% requirió tests de rutas excepcionales y validaciones defensivas que rara vez se ejecutan en producción. Fue

necesario priorizar: ¿cobertura alta o tests de alto valor funcional? La decisión fue enfocarse en cobertura de métodos público (APIs) primero.

Sincronización de Hallazgos: Las herramientas no siempre estaban alineadas: ZAP encontraba vulnerabilidades que SpotBugs no detectaba (son análisis ortogonales). Requirió matriz de trazabilidad para asegurar que cada hallazgo fuera correctamente clasificado y mitigado.

Aprendizajes Clave y Mejores Prácticas

1. Shift-Left Security es Crítico: Identificar vulnerabilidades durante desarrollo (análisis estático) es exponencialmente más barato que en producción. Los 15 bugs en SpotBugs fueron fixeados en horas; un crash en producción hubiera costado exponencialmente más en SLA e impacto reputacional.
2. Layered Defense (Defensa en Profundidad): Ninguna herramienta es perfecta. SpotBugs perdió XSS, ZAP no detectó null pointers, Dependency-Check depende de bases de datos actualizadas. La combinación de múltiples herramientas resulta en cobertura significativamente superior.
3. Contexto y Experiencia Importan: Interpretar hallazgos requiere experiencia en frameworks, patrones de seguridad y arquitectura. Un developer junior interpretaría como "8 críticas" los falsos positivos de SpotBugs. La capacitación del equipo es tan importante como las herramientas.
4. Automatización en CI/CD: Las correcciones fueron manuales esta vez, pero la verdadera ganancia viene de embebidas automáticamente en el pipeline CI/CD. Cada commit debería disparar estos análisis, fallando el build si hallazgos críticos son introducidos. Esto es especialmente importante para prevenir regresiones.

5. Documentación de Excepciones: No todas las alertas deben ser fijeadas. Algunas son falsos positivos legítimos (como Spring injection patterns). Estas requieren documentación explícita con justificación, creando "suppression lists" versionadas en el repo.